Problem Solving Short Note

Contents

1 Computational Thinking

Computational Thinking (CT) is a problem-solving process that involves breaking down complex problems systematically to develop solutions that a computer or human can execute. CT ultimately produces an algorithm a precise, replicable series of steps.

1.1 Pillars of Computational Thinking

- **Decomposition**
  - Breaking bigger problem into smaller, more manageable sub problems, each of which can be solved individually.

    Ex: Creating an app: What it looks like, target audience, graphics, audio, navigation, testing, distribution.
- **Pattern Recognition**
  - Identifying similarities, differences, trends or repeated structures across decomposed sub-problems to apply common solution efficiently.

    Ex: Netflix recommendations, clickstream analysis, disease outbreak detection
- **Abstraction**
  - Focusing on essential details and ignoring irrelevant information to simplify the problem representation.

    Ex: A road map: shows roads, cities, and rail – omits road width, exact geography

    Levels of abstraction in computing: Hardware → OS → Language → Application
- **Algorithmic Thinking**
  - Design a precise, step by step, replicable process that accepts defined inputs and produces defined output.
  - Properties of a good algorithm:
    - Purposeful : Solves a specific problem.
    - Describable : Can be communicated properly.
    - Replicable : Produce consistent results.
    - Finite : Terminate in finite number of steps.
- **Generalization**
- **Evaluation**

1.2 Steps of Computational Thinking

- **Problem Specification**
  - Analyze and state the problem precisely; define criteria for the solution using decomposition, abstraction, and pattern recognition.
  - Decomposition, Pattern Recognition, Abstraction
- **Algorithmic Expression**
  - Design the algorithm with appropriate data representations using flowcharts, pseudocode, or diagrams.
  - Algorithmic Thinking
- **Solution Implementation** & **Evaluation**
  - Implement, test, debug, and check if the solution generalizes to related problems.
  - Generalization and Evaluation

1.3 Programming Paradigms

- **Imperative Programming**
  - Gives the step by step instruction on how to solve a problem.

    Ex: C, Java, Pascal, Python
    - **Procedural Programming**
      - Organizes programs into procedures or functions for modular programming.
    - **Structured Programming**
      - Organizes programs into clear logical hierarchy.
    - **Object Oriented Programming**
      - Uses objects and classes to model real-world entities.
    - **Event Driven Programming**
      - The flow of the program is dictated by external events or signals.
- **Declarative Programming**
  - Describe the task to be done. The language figure out the way to achieve it.

    Ex: SQL, Prolog, Python
    - **Logic Programming**
      - Based on logical rules and facts for problem solving.
    - **Functional Programming**
      - Uses mathematical functions and immutable data.

2 Analysis of Algorithms

- The most straightforward reason for analyzing an algorithm is to discover its characteristics in order to evaluate its suitability for various applications or compare it with other algorithms for the same application.
- **Basic Operation** is the dominant operation whose count determines the algorithm's time complexity. It is the most frequently executed operation.
- Three cases of analysis :
  - **Best Case or Lower Bound** :
    - This refers to the scenario where the algorithm performs its task under the most favorable conditions.
    - Guaranteeing a lower bound on an algorithm does not provide any information as in the worst case, an algorithm may take years to run.
    - Notation : $B(n)$, $\Omega(n)$, $Big-\Omega$
    - Asymptotic Notation :

      $g(n) \in \Omega(f(n))$ if there exist constants $c > 0$ and $N \geq 0$ such that:

      $$g(n) \geq c.f(n) \ for\ all\ n \geq N$$

      Meaning: $g(n)$ grows at least as fast as $f(n)$ eventually.
  - **Average Case or Tight Bound** :
    - In average case analysis, we take all possible inputs and calculate the computing time for all of the inputs. Therefore it analyses the realistic performance
    - The usefulness of average case analysis can be limited if the actual input distribution significantly differs from the assumed distribution, or if the distribution of inputs is unknown.
    - Notation : $A(n)$, $\Theta(n)$, $Big-\Theta$
    - Asymptotic Notation :

      $g(n) \in \Theta(f(n))$ if there exist constants $c, d > 0$ and $N \geq 0$ such that:

      $$c.f(n) \leq g(n) \leq d.f(n) \ for\ all\ n \geq N$$

      Meaning: $g(n)$ grows at exactly the same rate as $f(n)$ eventually.
  - **Worst Case or Upper Bound** :
    - This refers to the scenario where the algorithm performs its task under the least favorable conditions.
    - This type of analysis examines the maximum number of operations an algorithm might perform or the maximum amount of resources it might require.
    - Notation : $W(n)$, $O(n)$, $Big-O$, $Small-o$
    - Asymptotic Notation :

      $g(n) \in O(f(n))$ if there exist constants $c > 0$ and $N \geq 0$ such that:

      $$g(n) \leq c.f(n) \ for\ all\ n \geq N$$

      Meaning for $g(n) \in O(f(n))$ (Big-O or Upper Bound): $g(n)$ does not grow faster than $f(n)$ eventually.

      Meaning for $g(n) \in o(f(n))$ (Small-o or Strict Upper Bound): $g(n)$ grows strictly slower than $f(n)$ eventually.
- Complexity Hierarchy :

$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$

2.1 Time Complexity

- The analysis is the determination of how many times the basic operation is done for each value of the input size.
- Estimation of total CPU computations required to execute an algorithm.

Time complexity of an algorithm	=	Total No. of CPU Computation	=	Sum of frequency count of each instruction of an algorithm

2.2 Space Complexity

- The analysis is the determination of how much memory (storage) an algorithm requires as a function of its input size, specifically focusing on the total amount of space used by the algorithm during its execution.
- Estimation of main memory space required to execute an algorithm.

Space complexity of an algorithm	=	Size of the Source code. (constant)	+	Size of the simple variables. (constant)	+	Size of data structures.	+	Size of call stack for recursive algorithms.

2.3 Commonly Used Summations

$1 + 2 + 3 + ... + n$	=	$\frac{n(n+1)}{2}$	=	$\Theta(n^2)$
$1^2 + 2^2 + 3^2 + ... + n^2$	=	$\frac{n(n+1)(2n+1)}{6}$	=	$\Theta(n^3)$
$1^3 + 2^3 + 3^3 + ... + n^3$	=	$\frac{n^2(n+1)^2}{4}$	=	$\Theta(n^4)$
$\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{n}$	=	$\log_e n$	=	$\Theta(\log n)$
${}^nC_0 + {}^nC_1 + {}^nC_2 + ... + {}^nC_n$	=	2^n	=	$\Theta(2^n)$

2.4 Problems with Loops

2.4.1 Single Loops

- **Constant Increment**

```
for (i from 1 to n; i = i + 1)
    print(i)
```

i = 1, 2, 3, 4, ... , n

Times the program ran : n

Time Complexity : $O(n)$

```
for (i from 1 to n; i = i + 2)
    print(i)
```

i = 1, 1 + 2 * 2, 1 + 3 * 2, 1 + 4 * 2, ... , 1 + k * 2

i = 1 + 0 * 2, 1 + 2 * 2, 1 + 3 * 2, 1 + 4 * 2, ... , 1 + k * 2

1 + k * 2 = n $\rightarrow k = \frac{n-1}{2}$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\frac{n-1}{2} + 1$

Time Complexity : $O(n)$

```
for (i from 1 to n; i = i + 10)
    print(i)
```

i = 1, 1 + 2 * 10, 1 + 3 * 10, 1 + 4 * 10, ... , 1 + k * 10

i = 1 + 0 * 10, 1 + 2 * 10, 1 + 3 * 10, 1 + 4 * 10, ... , 1 + k * 10

1 + k * 10 = n $\rightarrow k = \frac{n-1}{10}$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\frac{n-1}{10} + 1$

Time Complexity : $O(n)$

- **Constant Decrement**

```
for (i from n to 1; i = i - 1)
    print(i)
```

i = n, n - 1, n - 2, ... , 1

Times the program ran : n

Time Complexity : $O(n)$

```
for (i from n to 1; i = i - 2)
    print(i)
```

i = n, n - 2, n - 2 * 2, n - 3 * 2, ... , n - k * 2

i = n - 0 * 2, n - 1 * 2, n - 2 * 2, n - 3 * 2, ... , n - k * 2

n - k * 2 = 1 $\rightarrow k = \frac{n-1}{2}$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\frac{n-1}{2} + 1$

Time Complexity : $O(n)$

```
for (i from n to 1; i = i - 10)
    print(i)
```

i = n, n - 10, n - 2 * 10, n - 3 * 10, ... , n - k * 10

i = n - 0 * 10, n - 1 * 10, n - 2 * 10, n - 3 * 10, ... , n - k * 10

n - k * 10 = 1 $\rightarrow k = \frac{n-1}{10}$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\frac{n-1}{10} + 1$

Time Complexity : $O(n)$

- **Multiplying the Update Statement**

```
for (i from 1 to n; i = i * 5)
    print(i)
```

i = 1, 5, 5^2, 5^3, ... , 5^k

i = 5^0, 5^1, 5^2, 5^3, ... , 5^k

5^k = n $\rightarrow k = \log_5 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $log_5 n + 1$

Time Complexity : $O(\log n)$

- **Dividing the Update Statement**

```
for (i from n to 1; i = i / 5)
    print(i)
```

i = n, n/5, $n/5^2$, $n/5^3$, ... , $n/5^k$

i = $n/5^0$, $n/5^1$, $n/5^2$, $n/5^3$, ... , $n/5^k$

$\frac{n}{5^k} = 1 \rightarrow k = \log_5 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $log_5 n + 1$

Time Complexity : $O(\log n)$

- **Power in the Update Statement**

```
for (i from 3 to n; i = i^2)
    print(i)
```

i = 3, 3^2, 3^{2^2}, 3^{2^3}, ... , 3^{2^k}

i = 3^{2^0}, 3^{2^1}, 3^{2^2}, 3^{2^3}, ... , 3^{2^k}

3^{2^k} = n $\rightarrow 2^k = \log_3 n \rightarrow k = \log_2 \log_3 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\log_2 \log_3 n + 1$

Time Complexity : $O(\log \log n)$

```
for (i from n to 5; i = i^(1/2))
    print(i)
```

i = n, $n^{\frac{1}{2}}$, $n^{(\frac{1}{2})^2}$, $n^{(\frac{1}{2})^3}$, ... , $n^{(\frac{1}{2})^k}$

i = $n^{(\frac{1}{2})^0}$, $n^{(\frac{1}{2})^1}$, $n^{(\frac{1}{2})^2}$, $n^{(\frac{1}{2})^3}$, ... , $n^{(\frac{1}{2})^k}$

$n^{(\frac{1}{2})^k} = 5 \rightarrow n^{\frac{1^k}{2^k}} = 5 \rightarrow n^{\frac{1}{2^k}} = 5 \rightarrow 1 = \log_5 n^{\frac{1}{2^k}} \rightarrow 1 = \frac{1}{2^k} \log_5 n \rightarrow 2^k = \log_5 n \rightarrow k = log_2 log_5 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\log_2 \log_5 n + 1$

Time Complexity : $O(\log \log n)$

- **Conditional Statements**

```
for (i from 1 to 2^n; i = i + 1)
    print(i)
```

i = 1, 2, 3, 4, ... , k

k = 2^n

Times the program ran : 2^n

Time Complexity : $O(2^n)$

```
for (i from 2 to 2^n; i = i^2)
    print(i)
```

i = 2, 2^2, 2^{2^2}, 2^{2^3}, ... , 2^{2^k}

i = 2^{2^0}, 2^{2^1}, 2^{2^2}, 2^{2^3}, ... , 2^{2^k}

$2^{2^k} = 2^n \rightarrow 2^k = n \rightarrow k = \log_2 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $\log_2 n + 1$

Time Complexity : $O(\log n)$

```
for (i from 1 to n^(1/2); i = i + 1)
    print(i)
```

i = 1, 2, 3, 4, ..., k

k = $\sqrt{n}$

Times the program ran : $\sqrt{n}$

Time Complexity : $O(\sqrt{n})$

```
for (i from n^2 to 1; i = i / 2)
    print(i)
```

i = n^2, $\frac{n^2}{2}$, $\frac{n^2}{2^2}$, $\frac{n^2}{2^3}$, ... , , $\frac{n^2}{2^k}$

i = $\frac{n^2}{2^0}$, $\frac{n^2}{2^1}$, $\frac{n^2}{2^2}$, $\frac{n^2}{2^3}$, ... , , $\frac{n^2}{2^k}$

$\frac{n^2}{2^k} = 1 \rightarrow n^2 = 2^k \rightarrow \log_2 n^2 = \log_2 2^k \rightarrow k = 2\log_2 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $k = 2\log_2 n + 1$

Time Complexity : $O(\log n)$

```
for (i from 5^n to 5; i = i^(1/5))
    print(i)
```

i = 5^n, $5^{n(\frac{1}{5})}$, $5^{n(\frac{1}{5})^2}$, $5^{n(\frac{1}{5})^3}$, ... , , $5^{n(\frac{1}{5})^k}$

i = $5^{n(\frac{1}{5})^0}$, $5^{n(\frac{1}{5})^1}$, $5^{n(\frac{1}{5})^2}$, $5^{n(\frac{1}{5})^3}$, ... , , $5^{n(\frac{1}{5})^k}$

$5^{n(\frac{1}{5})^k} = 5 \rightarrow n(\frac{1}{5})^k = 1 \rightarrow n(\frac{1}{5^k}) = 1 \rightarrow \frac{n}{5^k} = 1 \rightarrow 5^k = n \rightarrow k = log_5 n$

Program ran from 0 to k, therefore k + 1 times.

Times the program ran : $k = log_5 n + 1$

Time Complexity : $O(\log n)$

2.4.2 Nested Loops

- **Independent Loops**
- Inner loop variable is independent of the variable of outer loop.

```
for (i from 1 to n; i = i + 1)
    for (j from 1 to n; j = j + 1)
        print(i, j)
```

Inner loop runs : n times

Outer loop runs : n times

Total run time : $n * n = n^2$

Time Complexity : $O(n^2)$

```
for (i from 1 to n; i = i + 1)
    for (j from 1 to n; j = j + 1)
        for (k from 1 to n; k = k + 1)
            print(i, j, k)
```

Inner loop runs : n times

Middle loop runs : n times

Outer loop runs : n times

Total run time : $n * n * n = n^3$

Time Complexity : $O(n^3)$

```
for (i from 3 to n; i = i^3)
    for (j from n to 1; j = j / 2)
        for (k from 1 to n; k = k * 4)
            print(i, j, k)
```

Inner loop runs : $\log_4 n$ times

Middle loop runs : $\log_2 n$ times

Outer loop runs : $\log_3 \log_3 n$ times

Total run time : $\log_4 n * \log_2 n * \log_3 \log_3 n = \log_4 n \log_2 n \log_3 \log_3 n$

Time Complexity : $O(\log^2 n . \log \log n)$

- **Dependent Loops**
- Inner loop variable is dependent on the variable of the outer loop variable

```
for (i from 1 to n; i = i + 1)
    for (j from 1 to n; j = j + i)
        print(i, j)
```

Outer loop runs : n times

$i = 1 \rightarrow$ for (j from 1 to n; j = j + 1) $\rightarrow$ Runs n times

$i = 2 \rightarrow$ for (j from 1 to n; j = j + 2) $\rightarrow$ Runs n/2 times

$i = 3 \rightarrow$ for (j from 1 to n; j = j + 3) $\rightarrow$ Runs n/3 times

$i = n \rightarrow$ for (j from 1 to n; j = j + n) $\rightarrow$ Runs n/n times

Total = $n + \frac{n}{2} + \frac{n}{3} + ... + \frac{n}{n}$

Total = $n\left[1 + \frac{1}{2} + \frac{1}{3} + ... + + \frac{1}{n}\right]$

Total = $n * \log_e n$

Time Complexity : $O(n \log n)$

```
for (i from 1 to n; i = i + 1)
    for (j from 1 to i; j = j + 1)
        for (k from 1 to j; k = k + 1)
            print(i, j, k)
```

Outer loop runs : n times

i = 1 | j from 1 to 1 | k from 1 to 1 $\rightarrow$ Runs 1 times

i = 1 | j from 1 to 2 | j = 1 then k from 1 to 1 & j = 2 then k from 1 to 2 $\rightarrow$ Runs (1 + 2) times

i = 1 | j from 1 to 3 | j = 1 then k from 1 to 1 & j = 2 then k from 1 to 2 & j = 3 then k from 1 to 3 $\rightarrow$ Runs (1 + 2 + 3) times

i = n | j from 1 to n | j = 1 then k from 1 to 1 & j = 2 then k from 1 to 2 & j = 3 then k from 1 to 3 ... & j = n then k from 1 to n $\rightarrow$ Runs (1 + 2 + 3 + ... + n) times

Total = (1) + (1 + 2) + (1 + 2 + 3) + ... + (1 + 2 + 3 + ... + n)

Total = x1 + x2 + x3 + ... + xn

Total = $\sum_{i=1}^{n} xi \rightarrow \sum_{i=1}^{n} \frac{i(i+1)}{2} \rightarrow \frac{1}{2} \sum_{i=1}^{n} i(i+1) \rightarrow \frac{1}{2} \sum_{i=1}^{n} (i^2 + i)$

Total = $\frac{1}{2}\left[\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i\right]$

Total = $\frac{1}{2}\left[\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right]$

Time Complexity : $O(n^3)$

3 Abstractions

3.1 Abstract Data Types (ADTs)

- ADTs are like user defined data types which defines what should be done without specifying how to it works.

3.1.1 Lists

- An ordered collection where elements have a specific position (index).
- Core operations :
  - insert(index, element) : insert an element to the list at the given index.
  - delete(index) : delete an element at the given index from the list.
  - get(index) : get an element at a given index from the list.
  - size() : get the size of the list.
- Ex : A to-do list

3.1.2 Stacks

- A linear data structure which follows the **LIFO** principle.
- LIFO (Last In-First Out) principle means the last item you added to the collection is the first one to be removed (newest item first).
- Vertical (pilling up) data structure.
- Core operations :
  - push(element) : insert an element to the end of the stack.
  - pop() : delete the top element of the stack.
  - peek() : get the top element of the stack.
  - isEmpty() : check whether the stack is empty.
- Ex : A stack of physical books. You place a book on top (Last-In). When you need a book, you grab the one from the top first (First-Out).

3.1.3 Queues

- A linear data structure which follows the **FIFO** principle.
- FIFO (First In-First Out) principle means the first item you added to the collection is the first one to be removed (oldest item first).
- Horizontal (lining up) data structure.
- Core operations :
  - enqueue(element) : insert an element to the end of the queue.
  - dequeue() : delete the first element of the queue.
  - peek() : get the first element of the queue.
  - isEmpty() : check whether the queue is empty.
- Ex : A line at a grocery store. The first person to join the line (First-In) is the first person to reach the cashier and leave (First-Out).

3.2 Control Abstraction

3.2.1 Iteration

- Iteration is the process of repeatedly executing a set of instructions or a block of code until a specific condition is met or a set goal is achieved.
- Loops like for, while are used for the iterations process.

3.2.2 Recursion

- A function that calls itself with a smaller/simpler input until it reaches a base case (stopping condition).
- **Base Case** : the simplest instance that does not recurse (e.g. n = 0 or n = 1). Avoids infinite recursion.
- **Recursive Case** : breaks the problem into a smaller sub-problem and calls itself.
- Call Stack :
  - Recursion is managed by the call stack (LIFO – Last In, First Out)
  - Each call creates a new stack frame storing local variables and return address
  - Deep recursion can cause a stack overflow error
- Tracing Types in Recursion :
  - Table Tracing
  - Tree Tracing
- Advantages of Recursion :
  - Understanding types of recursion can help in designing efficient and elegant recursive algorithms while considering the potential trade-offs
  - Recursion can simplify complex problems by breaking them down into smaller, more manageable pieces.
  - Recursive code can be more readable and easier to understand than iterative code.
  - Recursion is essential for some algorithms and data structures.
  - Also with recursion, we can reduce the length of code.
- Disadvantages of Recursion :
  - Recursion can be less efficient than iterative solutions in terms of memory and performance.
  - Recursive functions can be more challenging to debug and understand than iterative solutions.
  - Recursion can lead to stack overflow errors if the recursion depth is too high.
- **Direct Recursion** :
  - Function calls itself directly
  - **Head Recursion** :
    - Recursive call is the first statement; work done on the way back

```
function head(n):
    if n == 0:
        return n
    head(n - 1)
    print(n)    // Work is done after return
```

    - Output for head(5): 1 2 3 4 5

- **Tail Recursion** :
  - Recursive call is the last statement; work done before the call

```
function tail(n):
    if n == 0:
        return n
    print(n)    // Work is done before return
    tail(n - 1)
```

  - Output for tail(5): 5 4 3 2 1
- **Tree Recursion** :
  - Function calls itself more than once per invocation

```
function fibseq(n):
    if n <= 1:
        return n
    return fibseq(n - 1) + fibseq(n - 2)
```

- **Nested Recursion** :
  - Recursive call passes a recursive call as an argument

```
function nested(n):
    if n > 100:
        return n - 10
    return nested(nested(n + 11))
```

- **Indirect Recursion** :
  - Function calls another function that eventually calls the original function and it forms a cycle.

```
function func_1(n):
    if n > 1:
        print(n)
        return func_2(n / 2)
function func_2(n):
    if n > 0:
        print(n)
        return func_2(n - 1)
```

3.2.3 Recursion Vs. Iteration

- Both recursion and iteration are fundamental techniques in programming used to execute a set of instructions repeatedly.
- While they can often be used to solve the same problems, they differ significantly in their approach, memory usage, and performance.

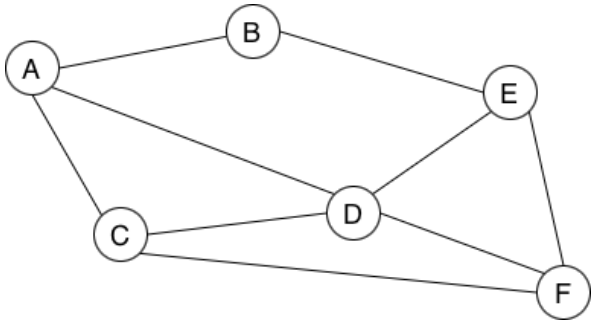
- Summary :
  - Algorithms involving trees, graphs, or divide-and-conquer strategies (like Quicksort or Merge Sort) are much easier to implement and read using recursion.
  - For complex algorithms, recursive code is often much cleaner and mirrors the mathematical definition of the problem.
  - If your application is performance-sensitive, iteration is usually preferred because it avoids the overhead of repeated function calls.
  - Recursion can lead to a Stack Overflow Error if the recursion depth is too high. Iteration uses a fixed amount of memory regardless of the number of repetitions.

Feature	Recursion	Iteration
Basic Mechanism	A function calls itself.	Uses loops.
Termination	Controlled with a base case.	Controlled with a loop condition.
State	Maintained in the call stack.	Maintained in local variables.
Memory	Uses more memory (for call stack).	Uses less memory.
Performance	Can be slower due to function call overhead.	Generally faster.
Code Size	Typically shorter and more elegant.	Typically longer and more verbose.

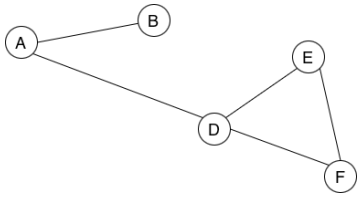
3.3 Architectural / Model Abstraction

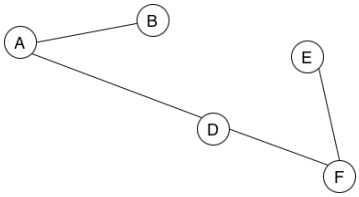
3.3.1 Graphs

- A non-linear data structure that consists of two primary components.
- Nodes (Vertices) : These are the fundamental units or points in the graph (often represented as circles).
- Edges (Arcs) : These are the connections or relationships between pairs of vertices (represented as lines).

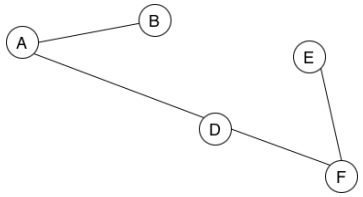


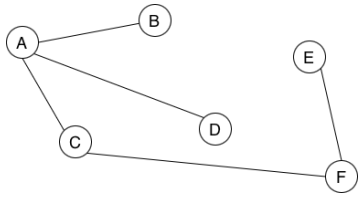
- **Subgraph** :
  - Subgraph is a graph whose vertices and edges are a subset of the original graph.



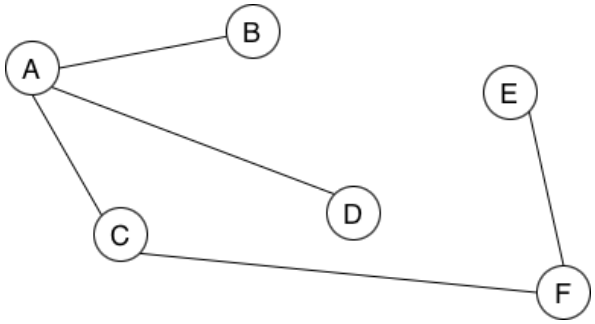


- **Trees** :
  - Tree is a subgraph where the vertices do not circuit with the edges.
  - In the above two images only the second one is a tree.
  - If a tree has n vertices it should only has n - 1 edges.



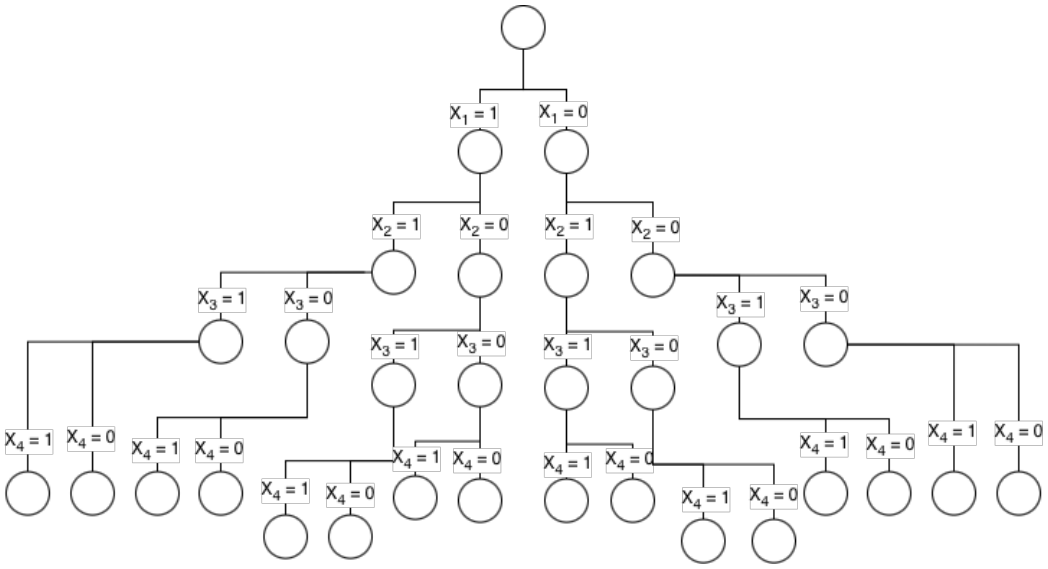


- **Spanning Trees** :
  - Spanning tree is a tree where it has all the vertices of the original graph.



3.3.2 State Space

- A state space tree is a mathematical representation of a search problem.
- You can use state space trees to show an abstraction of all the possible scenarios in a certain problem.
- Nodes (vertices or points) represent an entity.
- Arcs (edges or arrows) represents successors. (results of the operations)
- Goal test is a boolean test which tells whether the node is a goal node or not.
- **Fixed Tuple Form**
  - Shows the solution using binary values (0, 1).
  - Every path from the root to a leaf has the same length.
    Ex: All the solutions for the problem = (x_1, x_2, x_3, x_4)
    Fixed Tuple Form depiction of a sub solution = (1, 0, 0, 1)

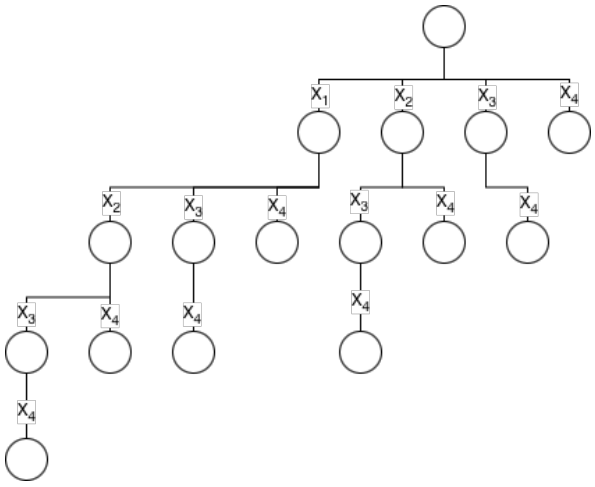


  - If it has 4 solutions, there will be 16 (2^4) leaf nodes.
  - Solution space is taken from root to leaf nodes.
  - Usually uses DFS to get the optimal solution.

- **Variable Tuple Form**
  - Shows the solution using variables.
  - The solution is represented by a vector of varying length, depending on the path taken.
  - This occurs in problems where the number of decisions is not predetermined or depends on previous choices.

    Ex: All the solutions for the problem = (x_1, x_2, x_3, x_4)

    Variable Tuple Form depiction of a sub solution = (x_1, x_4)



  - Solution space is taken from root to any node.
  - Usually uses BFS to get the optimal solution.

4 Problem Solving Approaches

4.1 Heuristics

- A heuristic is a rule of thumb or shortcut designed to solve a problem more quickly or when a perfect solution is impossible or impractical. It is an educated guess.

  Ex: A search (used in GPS navigation): Rather than calculating every possible road, it uses a heuristic to prioritize roads that are geographically closer to the destination. It finds a great route very fast, but might occasionally miss a slightly better one.

4.1.1 Trial and Error

- Trial and Error is a fundamental problem-solving method characterized by repeated, varied attempts which are continued until success, or until the experimenter stops.
- How it works :
  - Hypothesis: You guess a potential solution or action.
  - Test: You perform that action.
  - Evaluate: You observe the result. Does it work? Is it closer to your goal?
  - Adjust: Based on the feedback (success or failure), you formulate a new, slightly different hypothesis and repeat the process.
- Best for situation when the number of possible solutions is small, simply trying every option is often faster than designing a complex algorithm.
- Advantages :
  - It requires very little prior knowledge to start. It is excellent for discovery and exploration.
- Disadvantages :
  - It can be extremely inefficient. If you have thousands of possible combinations (like a complex password), guessing will take too long.

4.1.2 Means-End Analysis

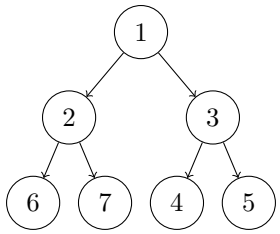
- Means-End Analysis is a problem-solving strategy that involves reducing the distance between your current state and your goal state.
- Think of it as a goal-oriented way of planning. Instead of looking at the whole mountain, you identify where you are, where you want to be, and what specific bridge you need to build to get there.
- How it works :
  - Define the Goal: Clearly state what the solved state looks like.
  - Define the Current State: Identify where you are now.
  - Identify the Difference: Determine the distance or gap between your current state and the goal.
  - Find a Sub-goal: Find an operator or a small step that reduces this difference.
  - Apply/Repeat: If the operator can be applied, do it. If not, set a new sub-goal to make the operator applicable.
- Advantages :
  - It is highly effective for complex, multi-step problems (like the Tower of Hanoi puzzle or general project management).
- Disadvantages :
  - It requires you to know your operators (the tools you have to solve the problem). If you don't know what actions are available to you, the method fails.

4.2 Algorithms

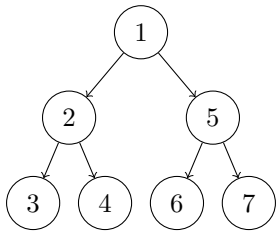
- An algorithm is a finite, well-defined sequence of instructions that, when followed correctly, is guaranteed to produce the correct output for a given input.

  Ex: Merge Sort: No matter the order of numbers you input, the algorithm will follow its structured steps and always return the array in perfect, sorted order.

- **Search Strategies** :
  - **BFS (Breadth First Search)** :
    - BFS explores a graph or tree level-by-level, starting from a chosen root or source node.
    - It visits all neighbors of the current node before moving on to the next level of neighbors.
    - A queue is used to manage nodes that need to be visited.



  - **DFS (Depth First Search)** :
    - DFS explores a graph or tree by going as deep as possible along each branch before backtracking.
    - It follows a single path to its end (a leaf or a node with no unvisited neighbors) before retreating to explore the next available branch.
    - A stack is used to keep track of the path and to facilitate backtracking.



  - **Best First Search** :
    - Best First search explores a graph by expanding the most promising node chosen according to a specified rule.
    - Uses a priority queue to store nodes, so the most promising one will always be at the front.

4.2.1 Brute Force

- Brute force in computer science refers to a straightforward approach to problem-solving, directly addressing the problem's possible solutions without applying any strategic logic or established algorithms.
- How it works :
  - List: Generate every possible configuration or value that could solve the problem.
  - Test: Check each generated candidate to see if it is the solution.
  - Result: Stop when the solution is found, or when all possibilities have been exhausted.
- Advantages :
  - Simplicity and ease of implementation.
  - Guaranteed to find a solution if it exists.
  - Useful for solving small instances or one-off problems.
  - Provides a baseline for evaluating more optimized algorithms.
- Disadvantages :
  - Poor scalability to large problem sizes.
  - Inefficient use of computational resources.
  - May be too slow for real-time or interactive applications.
  - Does not exploit problem-specific structure or heuristics.
- Optimizing Brute Force :
  - Prune the search space by eliminating infeasible solutions early.
  - Use heuristics to guide the search towards promising areas.
  - Memoize or cache intermediate results to avoid redundant work.
  - Parallelize independent subproblems to utilize multiple cores or machines.
- Applications :
  - Bioinformatics: Comparing DNA sequences to find mutations or aligning protein structures.
  - Puzzle solving: Exploring all possible moves in chess, sudoku, or Rubik's cube.
  - Generating test cases: Exhaustively testing all input combinations to find bugs in software.

4.2.2 Divide and Conquer

- When you divide the original problem into two smaller problem, solve them & combine the results, that is divide & conquer.
- How it works :
  - Divide: Break the original problem into a set of smaller sub-problems.
  - Conquer: Solve the sub-problems recursively. If the sub-problems are small enough (base case), solve them directly.
  - Combine: Merge the solutions of the sub-problems into the solution for the original problem.
- Advantages :
  - It often transforms inefficient $O(n^2)$ algorithms into much faster $O(n \log n)$ solutions.
  - Since sub-problems are independent, they can be processed simultaneously on different processors, making it ideal for multi-core computing.
  - By focusing on smaller sub-problems, the data often fits better into the CPU's cache, which speeds up memory access.
- Disadvantages :
  - Recursion carries the cost of function calls (stack memory usage). For very small input sizes, a simple iterative approach is often faster.
  - It is harder to implement and debug compared to a simple Brute Force approach.
  - Deep recursion can lead to a Stack Overflow error if the input is too large and the recursion depth exceeds memory limits.

4.2.3 Dynamic Programming

- Dynamic Programming (DP) is a powerful algorithmic paradigm used to solve complex problems by breaking them down into simpler, overlapping sub-problems.
- It is essentially an optimization of the Divide and Conquer approach.
- Core philosophy of dynamic programming is solve once, remember always.
- Instead of re-calculating the solution to the same sub-problem, DP algorithms store the result in a table (or array) the first time they compute it.
- This process is called Memoization (top-down) or Tabulation (bottom-up) :
- **Memoization** :
  - You start with the main problem and break it down.
  - When you need a sub-problem's result, you check a memo (a dictionary or array).
  - If it's there, you use it; if not, you compute and store it.
- **Tabulation** :
  - You solve the smallest sub-problems first and use those results to build up to the main problem solution in a table.
  - This avoids the recursion stack entirely.
- How it works :
  - Define the State: Determine what information is needed to uniquely identify a subproblem.
  - Establish the Recurrence Relation: Find the mathematical formula that connects the current problem to its subproblems.
  - Identify the Base Case(s): Determine the smallest, non-decomposable inputs.
  - Choose Your Strategy: Decide whether to use Memoization (Top-Down) or Tabulation (Bottom-Up).
- Advantages :
  - It can turn an exponential time complexity algorithm ($O(2^n)$ to $O(n)$ or $O(n^2)$).
  - DP is guaranteed to find the absolute best (optimal) solution if the problem satisfies certain properties.
- Disadvantages :
  - Because you have to store the results of sub-problems, it can consume a large amount of memory.
  - It is often more difficult to conceptualize and design than Brute Force or standard Divide and Conquer.
- Application :
  - Fibonacci Sequence: The most common example. Calculating $F(n)$ requires $F(n-1)$ and $F(n-2)$. If you use pure recursion, you calculate the same lower Fibonacci numbers millions of times. DP stores them so each is calculated only once.
  - Knapsack Problem: Determining the most valuable items to pack in a bag with a weight limit.
  - Shortest Path Problems: Algorithms like Bellman-Ford or Floyd-Warshall use DP to find the shortest distance between nodes in a graph.

4.2.4 Greedy Algorithms

- A Greedy Algorithm is a problem-solving strategy that builds up a solution piece by piece.
- At each step, it makes the choice that looks best at that specific moment, without considering the bigger picture or future consequences.
- In other words, a greedy algorithm makes locally optimal choices with the hope of finding a globally optimal solution.
- Once a greedy algorithm makes a choice, it never revisits or changes that choice.
- How it works :
  - Sort: Sort all the available options based on their given weight.
  - Selection: Choose the best available option based on a specific criterion.
  - Feasibility: Check if the chosen option is valid according to the problem's constraints.
  - Completion: Add the selection to the solution set. Repeat until the problem is solved.
- Advantages :
  - Very simple to design.
  - Easy to implement.
  - Extremely fast (efficient).
- Disadvantages :
  - Do not always produce the best (optimal) result. Because they don't look ahead, they can get trapped in a solution that seems good initially but leads to a suboptimal outcome later.

4.2.5 Backtracking

- It is an exhaustive search method—essentially a smart way of trying all possible combinations to find the ones that satisfy a set of constraints.
- Like in trial and error, it chooses a path and go along with it until a valid solution is found.
- If a path leads to an invalid solution, the algorithm will abandon it (backtrack).
- The real power of backtracking lies in pruning.
- Pruning is the act of cutting off a branch of the search tree early because you know it won't yield a valid result.
- This prevents the algorithm from wasting time exploring thousands of dead-end combinations, like in brute force algorithms.
- Uses a state space tree to solve the problem using DFS because it is the most memory efficient way.
- How it works :
  - Choose: Pick an option for the next part of the solution.
  - Constraint Check: Determine if this choice violates any rules of the problem.
    - If it violates a rule, undo the choice and try a different option.
    - If it is valid, move to the next step.
  - Goal Check: If the current path leads to a complete solution, return it. If not, continue down the path.
  - Backtrack: If all options at a certain step have been exhausted and none led to a solution, return to the previous step and try a different option.
- Advantages :
  - If a solution exists, backtracking is guaranteed to find it.
  - Backtracking can be used to find all possible solutions to a problem, not just the best one.
  - The ability to prune or stop exploring a branch the moment a rule is broken makes it far more efficient than a naive brute-force search.
- Disadvantages :
  - Backtracking is generally exponential in time, often $O(2^n)$ or $O(n!)$.
  - If the problem has overlapping sub-problems, backtracking will re-compute the result for that state every single time.
  - The efficiency of backtracking relies heavily on the order in which you explore choices.
- Optimizing Backtracking :
  - Instead of just checking if a path is valid (does it break rules?), check if a path is promising (can it even lead to an optimal solution?).
  - Backtracking often re-calculates the same state multiple times. If your problem has overlapping subproblems, use a cache to store the results of states you have already visited.
  - When picking which piece of the solution to fill next, don't just pick the next one in line. Pick the one with the fewest paths left to eliminate the dead ends earlier.

4.2.6 Branch and Bound

- Used primarily for solving combinatorial optimization problems—tasks where you need to find the best (optimal) solution from a vast, discrete set of possibilities.
- Uses a state space tree to solve the problem using BFS to find the best and most optimal solution.
- Bounds :
  - A numerical value calculated for a certain node in the state space tree, that estimates the best possible outcome of that could be achieved by following that branch.
  - **Upper Bound (for minimization problems)** :
    - The currently available solution with minimum cost is saved here.
    - If the cost of current branch is higher than the upper bound it is discarded.
  - **Lower Bound (for maximization problems)** :
    - The currently available solution with maximum profit is saved here.
    - If the profit for current branch is lower than the lower bound it is discarded.
- **Least Cost Branch-Bound** : Follows Best-First search to find the node with the lowest estimated cost to reach the goal.
- How it works :
  - Branching: The main problem is broken into smaller subproblems, forming a tree-like structure where each node represents a partial solution.
  - Bounding: For each node, the algorithm computes an upper or lower bound on the best possible solution within that subproblem. If this bound is worse than the current best-known solution, the node is pruned.
  - Pruning: By eliminating branches that cannot yield better solutions than the current best, the algorithm reduces the number of computations, enhancing efficiency.
- Advantages :
  - It is guaranteed to find the optimal solution if the search is completed.
  - By pruning the branches that cannot give an optimal solution it reduce time significantly.
  - It can be applied to a wide range of NP-hard problems.
  - Using BFS often leads to finding a high-quality solution much earlier in the process.
- Disadvantages :
  - Using BFS causes large amount of memory consumption due to storing all the neighboring nodes in a queue.
  - In the worst-case scenario the algorithm must explore the entire state-space tree, resulting in an exponential time complexity of $O(2^n)$ or worse.
  - Calculating the bounds for every node can be time-consuming.
  - Very complicated to design.

4.3 Summary

4.3.1 Algorithm Vs. Heuristic

Feature	Algorithm	Heuristic
Key Characteristic	Predictability and completeness.	Efficiency and speed.
Accuracy	Guaranteed to be correct.	Approximate.
Efficiency	Can be slow (computationally expensive).	Fast and resource-efficient.
Goal	Find the Best solution.	Find a Good solution quickly.
Use Case	When precision is non-negotiable.	When time or resources are limited.

4.3.2 Heuristic Paradigm Comparison

Feature	Trial and Error	Means-End
Approach	Random and unsystematic	Systematic
Strategy	Relies of experimentation	Relies of minimizing the distance between current state and goal state
Efficiency	Lower	Higher

4.3.3 Algorithmic Paradigm Comparison

Paradigm	Optimally Guarantee	Time Complexity	Use Cases
Brute Force	Guaranteed	$O(n!)$ or $O(2^n)$	Small input sizes, baseline testing
Divide and Conquer	Problem-dependent	$O(\log n)$ or $O(n \log n)$	Sorting, large scale search
Dynamic-Programming	Guaranteed	$O(n^2)$ or $O(n^3)$	Pathfinding, knapsack, sequence alignment
Greedy Approach	No	$O(n)$ or $O(n \log n)$	Huffman sorting, Dijksta
Backtracking	Guaranteed	$O(2^n)$	Sudoku, N-Queens
Branch and Bound	Guaranteed	$O(2^n)$	Traveling salesperson

5 Problems

5.1 Fibonacci Sequence

- A sequence where each term is the sum of the two preceding terms
- Iterative Implementation :
  - Time Complexity : $O(n)$
  - Linear, much efficient.
- Recursive Implementation :
  - Time Complexity : $O(2^n)$
  - Exponential, inefficient due to repeated subproblems.
- Dynamic Programming Implementation :
  - Time Complexity : $O(n)$
  - Efficient that recursion since all the intermediate calculations are stored in an array.
- Uses of Fibonacci Sequence:
  - The ratio of consecutive Fibonacci numbers approaches the Golden Ratio $\varphi \approx 1.618$.
  - Tribonacci variant: each term is the sum of the three preceding terms.
  - Appears in nature: sunflower spirals, leaf arrangements, nautilus shells.

5.2 Searching Algorithms

- Searching algorithms are methods for retrieving a specific piece of information from a collection of data.
- Searching algorithms are generally categorized into two main types: **Sequential (Linear) Search** and **Interval (Binary) Search**.

5.2.1 Sequential (Linear) Search

- Starting from the first element, compare the target x with each element one by one until found or the list is exhausted.
- A **brute force** approach.
- Best case :
  - x is the first item in S.
  - Time Complexity : $O(1)$
- Worst case :
  - x is the last element in S or x is not in S.
  - Time Complexity : $O(n)$
- No sorting is required. Therefore use for small unsorted lists.

5.2.2 Interval (Binary) Search

- Repeatedly halve the search range: compare target with the midpoint, then search left or right half accordingly.
- Best case :
  - x is the middle item in S.
  - Time Complexity : $O(1)$
- Worst case :
  - x is the last middle element in S or x is not in S.
  - Time Complexity : $O(\log n)$
- Sorting is required. Therefore use for large sorted lists.

5.2.3 Summary

Feature	**Linear Search**	**Binary Search**
Data Requirement	No Requirement	Must be sorted
Time Complexity	$O(n)$	$O(\log n)$
Efficiency	Slow for large datasets	Extremely fast
Complexity	Simple logic	Slightly more complex

- Summary :
  - If list is randomly ordered, use Linear Search.
  - If list is sorted, almost always use Binary Search because it is mathematically much faster for large amounts of data.

5.3 Sorting Algorithms

- Sorting algorithms are methods used to organize data into a specific order (usually numerical or alphabetical).
- Because there are many ways to sort, different algorithms are better suited for different types of data or performance requirements.
- **Stability** of an algorithm means equal elements preserve their original relative order after sorting.

  Original: [(Alice, 85), (Bob, 92), (Charlie, 85), (David, 78)]

  Sorted: [(David, 78), (Alice, 85), (Charlie, 85), (Bob, 92)]

  Notice how the term (Alice, 85) still comes before (Charlie, 85). Therefore this is a stable sorting type.
- **In-place sorting** means sorting without any extra space requirement. This means that the algorithm does not need to create a new array to store the sorted elements.

5.3.1 Comparison-Based Sorting

- Most common sorting algorithms are comparison-based; they determine the order of elements by comparing them to one another using operators like $<$ or $>$.
- Simple/Slow Algorithms ($O(n^2)$) :
  - Best for very small datasets or nearly sorted data.
  - **Bubble Sort**: Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. It bubbles the largest elements to the end.
  - **Insertion Sort**: Builds the final sorted list one item at a time. It takes an element and inserts it into its correct position among the already-sorted elements.
  - **Selection Sort**: Repeatedly finds the minimum element from the unsorted part and puts it at the beginning.
- Efficient/Fast Algorithms ($O(n \log n)$) :
  - Best for large datasets.
  - **QuickSort**: Uses a divide-and-conquer approach by partitioning the array around a pivot element.
  - **Merge Sort**: Divides the list into halves, sorts each half recursively, and then merges the sorted halves back together.
  - **Heap Sort**: Uses a binary heap data structure to organize the data. It is efficient and has the advantage of having a guaranteed worst-case time complexity

5.3.2 Distribution Sorting

- These algorithms don't compare elements directly. Instead, they exploit the specific properties of the data (like the range of integers or characters).
- These can sometimes achieve linear time complexity ($O(n)$) under specific conditions.
- **Counting Sort**: Counts the number of occurrences of each unique element and uses arithmetic to calculate the position of each element in the output sequence.
- **Radix Sort**: Sorts numbers digit by digit, starting from the least significant digit to the most significant.
- **Bucket Sort**: Distributes elements into various buckets, which are then sorted individually.

5.3.3 Merge Sort

- Find the middle of the array and splitting it into two equal halves. It continues to recursively divide these sub-arrays until you are left with individual elements.
- The algorithm takes two sorted sub-arrays and merges them into a single, larger, sorted array.
- It does this by comparing the first elements of each sub-array and picking the smaller one to place in the new list.
- Merge sort is a **stable sorting algorithm**.
- Merge sort is a **not in-place sorting algorithms**.
- Merge sort is an application of **divide and conquer** approach.
- Best Case :
  - When the array is already sorted or nearly sorted.
  - $O(n \log n)$

- Average Case :
  - When the array is randomly ordered.
  - $O(n \log n)$
- Worst Case :
  - When the array is sorted in reverse order.
  - $O(n \log n)$
- Auxiliary Space :
  - Additional space is required for the temporary array used during merging.
  - $O(n)$
- Advantages :
  - Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
  - Merge sort has a worst-case time complexity of O(N log N) , which means it performs well even on large datasets.
  - The divide-and-conquer approach is straightforward.
  - We independently merge sub-arrays that makes it suitable for parallel processing.
- Disadvantages :
  - Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
  - Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
  - Slower than QuickSort in general. QuickSort is more cache friendly because it works in-place.
- Application :
  - Sorting large datasets
  - External sorting (when the dataset is too large to fit in memory)
  - It is a preferred algorithm for sorting Linked lists.
  - It can be easily parallelized as we can independently sort sub-arrays and then merge.

5.3.4 Quick Sort

- Select a pivot element from the array. This could be the first element, the last element, the middle element, or a random one.
- Set two pointers, one starting at the beginning of the array and one at the end.
- Move the left pointer forward until you find an element larger than the pivot. Move the right pointer backward until you find an element smaller than the pivot.
- Swap these two elements.
- Repeat this until the pointers cross. Finally, you swap the pivot into its correct sorted position (the point where everything to the left is smaller and everything to the right is larger).
- In the two sub arrays obtained (left and right part of the pivot element), continue the sorting steps above for each of them.
- Quick sort is **not a stable sorting algorithm**.
- Quick sort is **an in-place sorting algorithm**.

- Quick sort is an application of **divide and conquer** approach.
- Best Case :
  - When the pivot consistently divides the array into two nearly equal halves.
  - $O(n \log n)$
- Average Case :
  - When the array is randomly ordered.
  - $O(n \log n)$
- Worst Case :
  - When the pivot is consistently the smallest or largest element.
  - $O(n^2)$
- Auxiliary Space :
  - Additional space is required as stack space for the recursive function calls.
  - $O(\log n)$
- Advantages :
  - It is often faster in practice than other $O(n \log n)$ algorithms like Merge Sort or Heap Sort because its inner loop can be efficiently implemented on most architectures.
  - Since it is in-place, it does not require the additional $O(n)$ memory overhead that Merge Sort typically requires.
  - It has good locality of reference, meaning it accesses memory in a way that is highly compatible with modern CPU caches.
- Disadvantages :
  - Without proper pivot selection (like random selection), its performance can degrade significantly on specific types of data.
  - If you need to preserve the relative order of duplicate records, QuickSort is not the ideal choice.
- Application :
  - General Purpose Sorting: It is the default sorting algorithm in many standard libraries.
  - Large Datasets: Because it is in-place and cache-efficient, it is excellent for sorting arrays that fit into main memory.
  - Systems Programming: Preferred where memory overhead must be minimized.

5.3.5 Summary

Algorithm	**Best Case**	**Worst Case**	**Space Complexity**	**In-place?**	**Stable?**
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	No	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Yes	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Yes	No

- If you have a small dataset: Simple algorithms like Insertion Sort are often faster due to their low overhead and simplicity.
- If memory is extremely limited: Heap Sort or Insertion Sort are preferred as they are in-place ($O(1)$ extra space).
- If stability is a requirement: If you need to maintain the relative order of identical elements, Merge Sort or Insertion Sort are the standard choices.
- If you are dealing with massive, general-purpose data: QuickSort is typically the default choice in most programming language libraries due to its high efficiency and cache-friendliness.

5.4 Strassen's Algorithm

- Strassen's algorithm is a famous **divide and conquer** approach to matrix multiplication.
- The standard (naive) algorithm multiplies two 2x2 matrices using 8 multiplications.
- Strassen discovered that by cleverly rearranging the terms, you can compute the product using only 7 multiplications, at the cost of performing more additions and subtractions.
- Time complexity for standard way is $O(n^3)$ while for Strassen's algorithm is $O(n^{\log_2 7}) \approx O(n^{2.807})$
- How it works :
  - To multiply two n x n matrices A and B, you divide them into four sub-matrices of size n/2 x n/2
  - Keep dividing the matrices until you get 2x2 matrix.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

  - Instead of the standard 8 products, Strassen defines 7 intermediate products (M_1 through M_7):

    $M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$

    $M_2 = (A_{21} + A_{22})B_{11}$

    $M_3 = A_{11}(B_{12} - B_{22})$

    $M_4 = A_{22}(B_{21} - B_{11})$

    $M_5 = (A_{11} + A_{12})B_{22}$

    $M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$

    $M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$
  - The final quadrants of the result matrix C are then calculated using only additions and subtractions of these seven M values.

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

    $C_{11} = M_1 + M_4 - M_5 + M_7$

    $C_{12} = M_3 + M_5$

    $C_{21} = M_2 + M_4$

    $C_{22} = M_1 - M_2 + M_3 + M_6$

- Practical Usage :
  - It requires significant extra memory to store the intermediate sub-matrices (M_1 - M_7) and the temporary sums.
  - Strassen's algorithm is less numerically stable than the standard method. Floating-point errors can accumulate differently due to the additions and subtractions.
  - Modern CPUs are heavily optimized for standard, blocked matrix multiplication. Strassen's recursive, cache-unfriendly structure often makes it slower on actual hardware until the matrices reach a very large size.
  - Practical implementations usually use a hybrid approach: they use Strassen's for large matrices, but switch to the standard $O(n^3)$ algorithm once the sub-matrices become small enough to fit efficiently in the CPU cache.

5.5 Maximum Sub-array Sum

- An array is a list of numbers, stored in a contiguous blocks. a sub-array is a contiguous subset of the original array. sub-arrays cannot skip numbers.
- For a given array, maximum amount of sub-arrays can be made can be calculated as $\frac{n(n+1)}{2}$.

  Array :

-3	1	2

  Sub-arrays :

-3

-3	1

-3	1	2

1

1	2

2

- Maximum Sub-array Sum (MSS) is referred to as the sub-array with the maximum sum of the elements available in the sub-array, among all the sub-arrays.
- MSS can be found using both **Brute Force** approach, **Divide and Conquer** approach as well as the **Dynamic Programming** approach.

5.5.1 Brute Force Approach

- Time complexity $O(n^2)$.
- MSS for sub-arrays with all negative numbers :

-3	-5	-1	-4	-23

  MSS = -1 (The largest negative number from the sub-array)
- MSS for sub-arrays with all positive numbers :

3	1	2	5	11

  MSS = 22 (The sum of all the elements from the sub-array)
- MSS for sub-arrays with mixed numbers :

-3	1	2

Sub-arrays :

-3			sum = -3
-3	1		sum = -2
-3	1	2	sum = 0
1			sum = 1
1	2		sum = 3
2			sum = 2

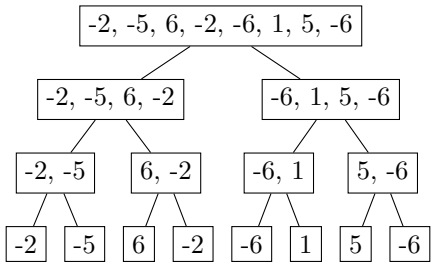
MSS = 3 (Maximum of all the sub-arrays)

5.5.2 Divide and Conquer Approach

- Time complexity $O(n \log n)$.

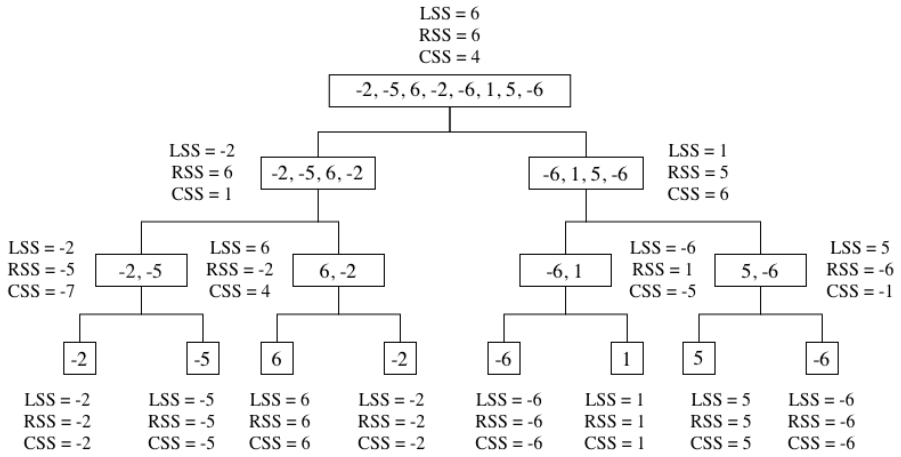
-2	-5	6	-2	-6	1	5	-6

- Keep dividing the array into two halves until it cannot be further divided.



- Find LSS (Left Sub-array Sum), RSS (Right Sub-Array Sum) and CSS (Cross Sub-array Sum).
- LSS :
  - MSS of the left part of the array (Highest number from LSS, RSS and CSS from the left part).
- RSS :
  - MSS of the right part of the array (Highest number from LSS, RSS and CSS from the right part).
- CSS :
  - Divide the array from middle.
  - Keep adding elements in the left part.
  - If the next element decreases the current sum, stop adding.
  - Continue the same for left part.
  - Add the two values together.

- For individual elements the LSS and RSS and CSS is just the same as the value of the element.



- Therefore MSS = 6 (Highest number from LSS, RSS and CSS from the left part)

5.5.3 Dynamic Programming Approach

- Time complexity $O(n)$.

A = | -2 | 1 | -3 | 4 | -1 | 2 | 1 | -5 | 4 |

- Maintain a solution array of sub-array sum.
- Add the current number to your existing sub-array sum.
- Get the current number and the sum of current number and the previously appended value to the sub-array sum.
- Append the highest number among them to the sub-array sum.
- After looping through the entire array, find the maximum of the solution array of sub-array sum.

Sub-array sum :

A[0] = -2

Sub-array sum : | -2 |

A[1] = 1, then append max(1, (-2 + 1)) to sub-array sum.

Sub-array sum : | -2 | 1 |

A[2] = -3, then append max(-3, (1 - 3)) to sub-array sum.

Sub-array sum : | -2 | 1 | -2 |

A[3] = 4, then append max(4, (-2 + 4)) to sub-array sum.

Sub-array sum : | -2 | 1 | -2 | 4 |

A[4] = -1, then append max(-1, (4 - 1)) to sub-array sum.

Sub-array sum : | -2 | 1 | -2 | 4 | 3 |

A[5] = 2, then append max(2, (3 + 2)) to sub-array sum.

Sub-array sum :

-2	1	-2	4	3	5

A[6] = 1, then append max(1, (5 + 1)) to sub-array sum.

Sub-array sum :

-2	1	-2	4	3	5	6

A[7] = -5, then append max(-5, (6 - 5)) to sub-array sum.

Sub-array sum :

-2	1	-2	4	3	5	6	1

A[8] = 4, then append max(4, (1 + 4)) to sub-array sum.

Sub-array sum :

-2	1	-2	4	3	5	6	1	5

Highest is sub-array sum is 6.

$\therefore$ MSS = 6

5.5.4 Kadane's Algorithm

- Time complexity $O(n)$.
- Similar to dynamic programming.
- Add the current number to your existing sub-array sum.
- If the current number is greater than the sum of the current number plus your previous streak, abandon the old streak and start a fresh sub-array right here.

A =

4	-1	2	1	-5	3

	Current Sum	Maximum
A[0] = 4	4	4
A[1] = -1	3	4
A[2] = 2	5	5
A[3] = 1	6	6
A[4] = -5	1	6
A[5] = 3	4	6

- MSS = 6

5.6 Coin Change Problem

- This problem can be solved in both **Greedy** and **Dynamic Programming** approach.
- Even though dynamic programming generates an always optimal solution, the speed of greedy algorithm is faster with a time complexity $O(n)$ while dynamic programming's time complexity is $O(\text{Amount}.n)$.
- Problem :
  - Given a set of coins and a target amount, find the minimum number of coins needed to make that total. Assume that there are infinite supply of coin.

5.6.1 Greedy Approach

- Sort coins in descending order (array of length n).
- Check if the largest coin fits into the remaining amount.
- If it fits, add it to your collection and subtract its value from the amount.
- Repeat until the target amount is zero.

When greedy is successful :	When greedy fails :
Making \$63 with standard coins (25, 10, 5, 1)	Making \$6 with standard coins (1, 3, 4)
Target = \$63 and Sorted coin list = (25, 10, 5, 1)	Target = \$6 and Sorted coin list = (4, 3, 1)
Pick \$25 (Remaining: \$38)	Pick \$4 (Remaining: \$2)
Pick \$25 (Remaining: \$13)	Pick \$1 (Remaining: \$1)
Pick \$10 (Remaining: \$03)	Pick \$1 (Remaining: \$0)
Pick \$01 (Remaining: \$02)	
Pick \$01 (Remaining: \$01)	
Pick \$01 (Remaining: \$00)	
Total number of coins = 6	Total number of coins = 3
This is the optimal solution.	But the optimal solution is 2 of \$3 coins.

- If the coins are arbitrary, the greedy choice can fail to find the minimum number of coins.

5.6.2 Dynamic Programming Approach

- Sort all the coin values in ascending order.
- Create a 2D table where columns represent target amounts (starting from 0) and rows represent the available coins.
- For every coin value, determine the minimum number of coins required to make each target amount.
- You check the minimum coins needed by either excluding the current coin or including it.
- As you progress through the rows, you combine different coins to find the most efficient (minimum) count for every amount until you reach your final target.

  Making \$6 with standard coins (1, 3, 4)

  Target = \$6 and Sorted coin list = (1, 3, 4)

	0	**1**	**2**	**3**	**4**	**5**	**6**
1	0	1	2	3	4	5	6
3	**0**	1	2	**1**	2	3	**2**
4	0	1	2	3	1	2	**2**

  Therefore the minimum no of coins is 2 (The last number in the table).

  The last 2 is coming from above, go one step above and take \$3 as a coin.

  That 2 is not coming from above, so go 3 steps left and take \$3 as a coin.

  The 1 is not coming from above, go 3 steps left. Its a 0 so all the coins are taken.

Therefore problem can be solved using 2 $3 coins.

- To solve the problem for any set of coins, dynamic programming generates a better solution by storing results of smaller sub-problems in a table.

5.7 Activity Selection Problem

- The Activity Selection Problem is where a **Greedy Algorithm** is not just an approximation—it is guaranteed to find the optimal (maximum) number of activities that can be performed by a single person or resource.
- Problem :
  - You are given a set of n activities, each with a start time and a finish time.
  - A person can only perform one activity at a time.
  - Select the maximum number of non-overlapping activities.
- How it works :
  - Sort: Sort all activities by their finish times in ascending order.
  - Select the first: Always select the first activity from the sorted list.
  - Iterate: For the remaining activities, select an activity only if its start time is greater than or equal to the finish time of the previously selected activity.

Example :

Activity	Start	End
A1	1	4
A2	3	5
A3	0	6
A4	5	7
A5	3	9
A6	5	9
A7	6	10
A8	8	11
A9	8	12
A10	2	14
A11	12	16

Sorted activities = (A1, A2, A3, A4, A5, A6, A7, A8, A9, A10, A11)

Selected activities = (A1)

A2 and A3 has a start time less than end time of A1.

Selected activities = (A1, A4)

A5, A6, and A7 has a start time less than end time of A4.

Selected activities = (A1, A4, A8)

A9 and A10 has a start time less than end time of A8.

Selected activities = (A1, A4, A8, A11)

- Unlike the Coin Change problem, where greedy could fail, the Activity Selection problem has Optimal Substructure and the Greedy Choice Property.
- The time complexity of greedy algorithm here is $O(n \log n)$ due to initial sorting.
- But it is still faster than solving this problem using dynamic programming which has a time complexity of $O(n^2)$ or higher.

5.8 Knapsack Problem

- The Knapsack Problem is a classic optimization challenge.
- There are two primary versions, and the strategy you use **Greedy** or **Dynamic Programming** depends entirely on which version you are solving.
- Problem :
  - You have a knapsack with a maximum weight capacity W.
  - You are presented with a set of n items, each having a specific weight and a value.
  - Your goal is to fill the knapsack to achieve the maximum total value without exceeding the weight limit.
  - Only 1 item from the same weight category is available to you.

5.8.1 The Fractional Knapsack Problem

- In this version, you are allowed to break items into smaller pieces (like bags of flour, gold dust, or liquids).
- How it works :
  - Calculate the value-to-weight ratio for every item.
  - Sort all items in descending order based on this ratio.
  - Fill the knapsack by taking as much of the item with the highest ratio as possible.
  - If the item fits completely, take it. If not, take a fraction of it to fill the remaining capacity.

  Example :

  Knapsack capacity = 5kg.

  Item A: 4kg, \$40 $\rightarrow$ ratio = 10

  Item B: 3kg, \$27 $\rightarrow$ ratio = 9

  Item C: 2kg, \$20 $\rightarrow$ ratio = 10

  Item list = (A, C, B)

  5kg is made with Item A and 1kg fraction from the Item C.

  Total = (4kg $\rightarrow$ \$40) + (1kg $\rightarrow \frac{\$20}{2} \rightarrow$ \$10)

  Total = \$50

  This is the optimal solution.
- Therefore the optimal solution can be obtained using greedy algorithm.
- Time complexity of $O(n \log n)$

5.8.2 The 0/1 Knapsack Problem

- In this version, you cannot break items. You must either take the whole item or leave it behind.
- How it works :
  - Calculate the value-to-weight ratio for every item.
  - Sort all items in descending order based on this ratio.
  - Fill the knapsack by taking as much of the item with the highest ratio as possible.
  - If the item fits completely, take it. If not, go with another ratio.

  Example :

  Knapsack capacity = 5kg.

  Item A: 4kg, \$40 → ratio = 10

  Item B: 3kg, \$27 → ratio = 9

  Item C: 2kg, \$20 → ratio = 10

  Item list = (A, C, B)

  5kg is made with Item A and no other item available to fill the remaining 1kg.

  Total = (4kg → \$40)

  Total = \$40

  This is not the optimal solution.

  In this case optimal solution can be made using Item B and Item C.

  Total = (3kg → \$27) + (2kg → \$20)

  Total = \$47

	0	**1**	**2**	**3**	**4**	**5**
(2 / 20)	**0**	0	**20**	20	20	20
(3 / 27)	0	0	**20**	27	27	**47**
(4 / 40)	0	0	20	27	40	**47**

  This solution can only be achieved by dynamic programming.
- Therefore the optimal solution can be obtained using dynamic programming.
- Time complexity of $O(n.W)$

5.9 Minimum Spanning Tree Problem

- Can be solved easily using the **Greedy** approach.
- In a weighted graph (where each edge has a cost or length), a Minimum Spanning Tree (MST) is a spanning tree where the sum of the weights of all edges is minimized.
- Because MSTs are foundational in network design (like building roads, laying cables, or designing server clusters), we have two famous greedy algorithms to solve them.
- There can be multiple minimum spanning trees if there are similar weights in the graph.

5.9.1 Kruskal's Minimum Spanning Tree Algorithm

- Focus on edges. So best for graphs with smaller amount of edges (sparse graphs).
- How it works :
  - First choose any edge with the minimum weight.
  - Continue to choose any edge with the minimum weight from those not yet selected without creating circuits.
  - Repeat until all the vertices are connected.

5.9.2 Prim's Minimum Spanning Tree Algorithm

- Focus on vertices. So best for graphs with larger amount of edges (dense graphs).
- How it works :
  - Select a random vertex.
  - Find the edge with minimum weighted edge that is connected to that vertex.
  - From all the vertices that are selected find the edge with minimum weight that is connected to those vertices, which leads to a vertex from those not yet selected without creating a circuit.
  - Repeat until all the vertices are connected.

5.10 Maze Problem

- Maze problem can be solved using **Backtracking** and **Branch and Bound** approachess.
- Problem :
  - You are given an NxN grid, and your goal is to find a path from a starting cell to a destination cell.
  - Some cells are open (passable), while others are blocked (walls).

5.10.1 Backtracking Approach

- Move: Try moving in a valid direction (Up, Down, Left, Right).
- Mark: Mark the current cell as visited so you don't walk in circles.
- Recursion: From the new cell, try to reach the destination recursively.
- Backtrack: If none of the directions lead to the destination, un-mark the current cell (reset it) and retreat to the previous position to try a different direction.
- Backtracking approach might not find the optimal path, but it can be used to find all the available paths since it uses very little memory.

5.10.2 Branch and Bound Approach

- Create an state space tree for all the possible moves.
- Calculate the current path length from start to current cell (l_1).
- Estimate the distance from start to destination to check for promising paths (l_2).
- Bound will be the total estimated length ($l_1 + l_2$)
- If the best possible path so far exceeds that, prune that path.
- Backtracking approach might not find the optimal path, but it can be used to find all the available paths since it uses very little memory.

5.11 Sudoku

- Can be solved easily using **Backtracking** algorithm.
- Problem :
  - Solve a sudoku board (9x9) with some values available at the beginning.
  - All the rows, columns and 3x3 sub-grids can only contain numbers through 1-9 without any duplications.
- How it works :
  - The algorithm scans the 9x9 grid to find a cell that has not been filled yet (usually represented by a 0). If it cannot find any empty cells, the puzzle is solved.
  - Once an empty cell is found, the algorithm attempts to place a number from 1 to 9 in that cell.
  - Before moving on, it checks if placing that number violates any Sudoku rules.
  - If the number is valid, the algorithm tentatively accepts it and recursively repeats the entire process from Step 2 for the next empty cell.
  - If placing a number leads to a dead end later on, it returns to the current cell, erases the guess (resets it to empty), and tries the next available number.
  - If the algorithm tries all numbers from 1 to 9 in a cell and none of them lead to a solution, it triggers a backtrack to the previous empty cell to change the number placed there.

5.12 N Queens Problem

- Can be solved easily using **Backtracking** algorithm.
- Problem :
  - There a N number of queens on a chess board of size NxN.
  - Queens should be placed so no two queens threaten each other.
  - So no queens should be on same row, column or diagonal.
  - Find all the possible placements of queens.
- How it works :
  - Place the queens on each column and each rows.

    Ex. 1st queen on 1st column 1st row, 2nd queen on 2nd column 2nd row and so on.
  - Compare all the placements considering attacks.

Ex. Place 4 queens on 4x4 chess board.

Q1			
	Q2		

Q2 cannot be placed there.

Q1			
		Q2	
	Q3		

Q3 cannot be placed there.

Q1			
		Q2	
			Q3

Q3 cannot be placed there.

Q1			
			Q2
	Q3		
		Q4	

Q4 cannot be placed there.

Q1			
			Q2
		Q3	

Q3 cannot be placed there.

	Q1		
Q2			

Q2 cannot be placed there.

	Q1		
		Q2	

Q2 cannot be placed there.

	Q1		
			Q2
Q3			
		Q4	

A possible solution.

	Q1		
			Q2
		Q3	

Q3 cannot be placed there.

		Q1	
Q2			
	Q3		

Q3 cannot be placed there.

		Q1	
Q2			
			Q3
	Q4		

A possible solution.

		Q1	
	Q2		

Q2 cannot be placed there.

		Q1	
			Q2
	Q3		

Q2 cannot be placed there.

			Q1
Q2			
	Q3		

Q3 cannot be placed there.

			Q1
Q2			
		Q3	
	Q4		

Q4 cannot be placed there.

			Q1
	Q2		
Q3			

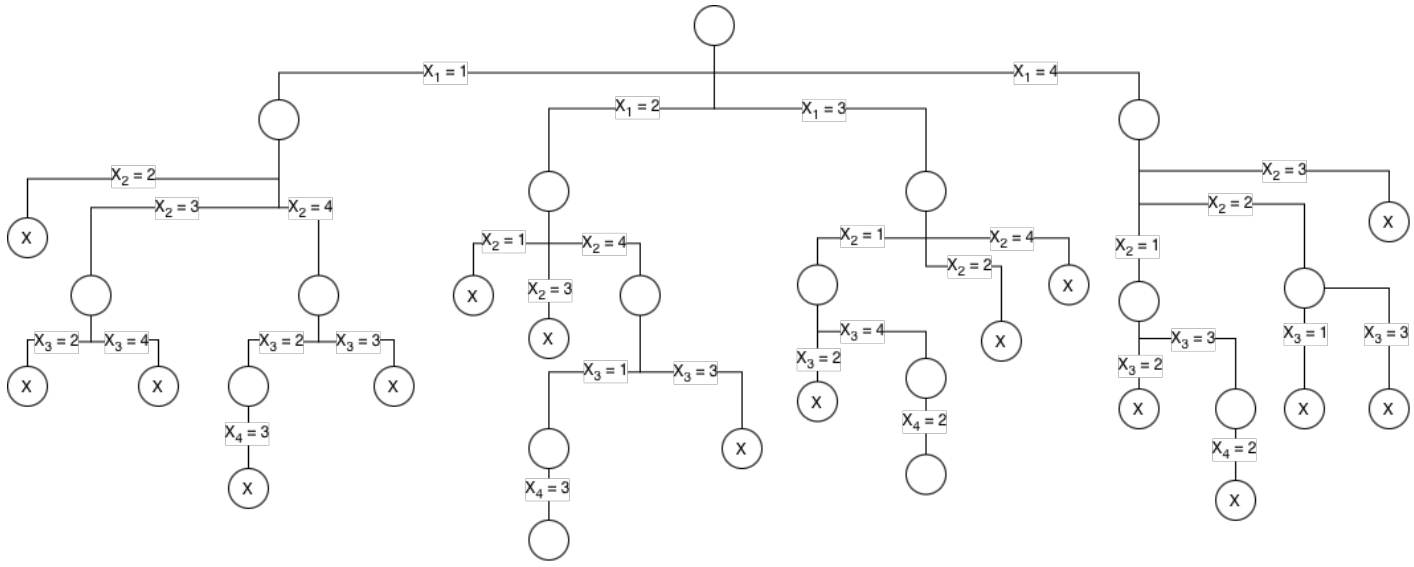
Q3 cannot be placed there.

			Q1
	Q2		
		Q3	

Q3 cannot be placed there.

			Q1
		Q2	

Q2 cannot be placed there.



Therefore the solutions are (2, 4, 1, 3) and (3, 1, 4, 2).

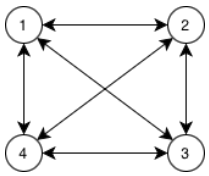
5.13 Hamiltonian Problem

- Starting from a vertex and visit all the vertices only once, generates a path and it is known as the Hamiltonian Path.
- If the Hamiltonian path connects back to the starting point to make a cycle, it is known as the Hamiltonian Cycle.
- Every Hamiltonian Cycle contains a Hamiltonian Path, but every Hamiltonian Path does not creates a Hamiltonian Cycle.
- If there is a pendent vertex (vertex that connected by only one edge) and an articulation point (where if the vertex is removed the graph will separate into pieces) on the graph, there is no Hamiltonian Path.
- **Backtracking** can be used to solve this.
- Problem : Find all the Hamiltonian Paths and Hamiltonian Cycles in a given graph.
- How it works :
  - Create an array of size n, where n is the no of vertices on the graph.
  - Select a vertex from the graph and set it as the first element of the array.
  - All the other elements should be set to zero.
  - Then continue selecting a vertex from the graph which has not already been selected and check whether their is an edge connecting it with the vertex selected before it.
  - If there exists such vertex, set it as the next element in array.
  - Continue until all the paths have been found.
  - To check for Hamiltonian Cycles add a condition for the last element in the array so it has an edge connected to the first element of the array.

5.14 Traveling Salesman Problem (TSP)

- Traveling Salesman Problem, can be solved using **Brute Force** approach, **Dynamic Programming** approach or the **Branch and Bound** approach.
- Problem :
  - A graph with each node is connected to each other bi-directionally is given with the cost of each route in an matrix.
  - The salesman need to go from a selected starting point, cover all the points and then return back to the starting point.
  - Find the best possible path with the lowest cost.

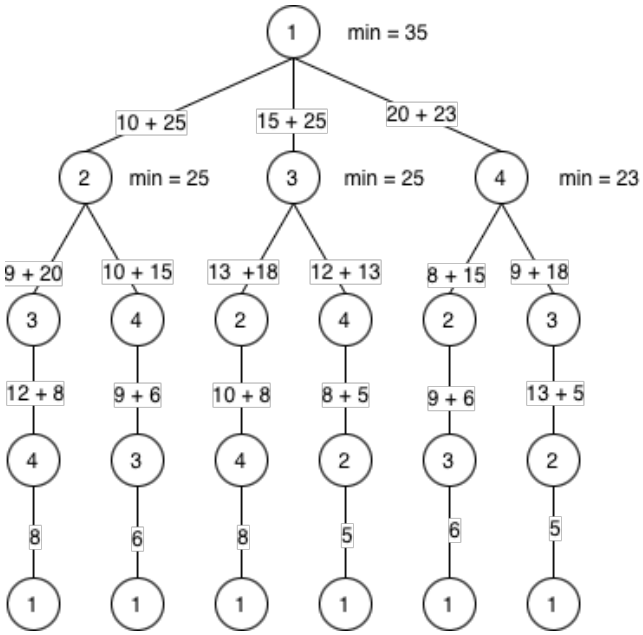
  Ex. Find most cost effective path of the bellow. Matrix A has the cost of all the edges.



$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{bmatrix} \end{array}$$

5.14.1 Brute Force Approach

- Draw the state space tree for all the routes.
- Calculate the cost of all the paths starting from the leaf nodes.
- Find the minimum of them.



- Therefore the optimal path is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$.

5.14.2 Dynamic Programming Approach

- Calculate the minimum value of the problem using the formula bellow.

$$g(i, S) = k \in S, \min\{C_{ik} + g(k, S - \{k\})\}$$

S = Set of nodes remaining to visit

k = Next node to visit

i = Current node

C_{1k} = Cost of visiting a node

- Start from the bottom layer of the problem.

$g(2, \phi) = 5$ $g(2, \{3\}) = 15$ $g(2, \{3, 4\}) = 25$ $g(1, \{2, 3, 4\}) = 35$

$g(3, \phi) = 6$ $g(2, \{4\}) = 18$ $g(3, \{2, 4\}) = 25$

$g(4, \phi) = 8$ $g(3, \{2\}) = 18$ $g(4, \{2, 3\}) = 23$

$g(3, \{4\}) = 20$

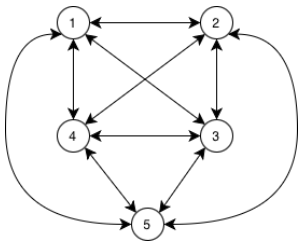
$g(4, \{2\}) = 13$

$g(4, \{3\}) = 15$

- Therefore the optimal path is $1 \to 2 \to 4 \to 3 \to 1$.

5.14.3 Branch and Bound Approach

Ex.



$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} \end{array}$$

- Get the minimum of each row in the matrix.

  10, 2, 2, 3, 4; Total cost of reduction = 10 + 2 + 2 + 3 + 4 = 21.

- Convert the matrix into the row reduced form.

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix} \end{array}$$

- Get the minimum of each column in the matrix.

  1, 0, 3, 0, 0; Total cost of reduction = 1 + 0 + 3 + 0 + 0 + 21 = 25.

- Convert the matrix into the column reduced form.

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix} \end{array}$$

- Total cost of reduction may be the lowest cost path. It may be larger than that too.
- The cost of root node (1 in here) will be the total cost of reduction.
- To go to the next place, make the values in second column and first row to infinity (Since we are going from 1 to 2 next).
- Also to avoid going back to 1 again make the relevant place to infinity too.

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix} \end{array}$$

- Check whether the matrix is reduced or not (Every column and row should contain a zero) and if not, convert it to reduced form and calculate the cost of reduction.
- Since matrix is already in reduced form, the total cost of reduction for node 2 is zero.
- Calculate the cost of next node as follows,

  Cost of node 2 = C(1, 2) + r + r_1

  C(1, 2) = Cost of going into 2 from 1 in the reduced matrix of the parent node

  r = Cost of reduction in node 1

  r = Additional reduction made when coming to node 2

  Cost of node 2 = 10 + 25 + 0 = 35

- Continue the cost for going from node 1 to 3

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix} \end{array}$$

Reduced form

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 11

Cost of node 3 = C(1, 3) + r + r_1

Cost of node 3 = 17 + 25 + 11 = 53

- Continue the cost for going from node 1 to 4

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 0

Cost of node 4 = C(1, 4) + r + r_1

Cost of node 4 = 0 + 25 + 0 = 25

- Continue the cost for going from node 1 to 5

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & 10 & 17 & 0 & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \end{array}$$

Reduced form

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & 10 & 17 & 0 & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 2 + 3 = 5

Cost of node 5 = C(1, 5) + r + r_1

Cost of node 5 = 1 + 25 + 5 = 31

- Since out of the available paths the minimum cost is to go from 1 to 4 (25).
- Therefore explore that branch next.
- Calculate the cost for going into node 4 to 6

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 0

Cost of node 6 = C(4, 2) + r + r_1

Cost of node 6 = 3 + 25 + 0 = 28

- Calculate the cost for going into node 4 to 7

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 3 & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Reduced form

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & \infty & 0 \\ \infty & 1 & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & 0 & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 11 + 2 = 13

Cost of node 7 = C(4, 3) + r + r_1

Cost of node 7 = 12 + 25 + 13 = 50

- Calculate the cost for going into node 4 to 8

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & \infty \\ 0 & 3 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty & \infty \end{bmatrix} \end{array}$$

Reduced form

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & 0 & \infty & \infty \\ 0 & 3 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 11

Cost of node 8 = C(4, 5) + r + r_1

Cost of node 8 = 0 + 25 + 11 = 36

- Out of all the nodes unexplored the minimum one is the node 6. So explore it next.
- Calculate the cost for going into node 6 to 9

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Reduced form

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 11 + 2 = 13

Cost of node 9 = C(2, 3) + r + r_1

Cost of node 9 = 11 + 28 + 13 = 52

- Calculate the cost for going into node 6 to 10

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 0

Cost of node 10 = C(2, 5) + r + r_1

Cost of node 10 = 0 + 28 + 0 = 28

- Calculate the cost for going into node 10 to 11

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Total cost of reduction = 0

Cost of node 11 = C(5, 3) + r + r_1

Cost of node 11 = 0 + 28 + 0 = 28

- Then set the upper bound equal to 28 (Maximum possible cost of the path).
- Then kill all the other nodes which costs more than the upper bound and check the ones that are less than the upper bound as the same way above.

  In this problem all the remaining nodes' costs are greater than upper bound.

  Therefore 28 is the least cost path which is $1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3$